

# A Lightweight, Perspective-Calibrated Multi-Robot Tracking and Heuristic Shot Analysis System for Competitive Mobile Robotics

Siddharth Ray\*

July 18, 2026

## Abstract

This paper presents the computer-vision core of AutoScout, a lightweight monocular video analysis system designed to extract tactical analytics from and virtually reconstruct *FIRST* Tech Challenge (FTC) match footage. The system is split into two coupled layers: a season-robust robot tracker that maps pixel data to four persistent trajectories in field coordinates, and a game-specific shot-analysis module that attaches scoring events to those trajectories. Operating under strict mobile pit constraints with low computational overhead, the pipeline rejects heavy deep-learning frameworks in favor of a deterministic, geometry-first approach. The core tracking framework combines homography-based perspective calibration, perspective-scaled foreground segmentation, and distance-transform splitting of merged silhouettes. Rather than relying on standard nearest-neighbor matching, AutoScout handles complex multi-robot collisions by modeling identity preservation as a temporary permutation problem. The result is a highly transparent, debuggable, and computationally efficient tracker that democratizes elite-level spatial data analytics for resource-constrained teams.

## 1 Introduction

During the 2023 *FIRST* Tech Challenge (FTC) World Championships, teams faced one of the most strategically complex games in modern FTC history at the highest competitive level. The tournament highlighted how critical data-driven strategy has become, culminating in a shocking playoff run in the Franklin Division by the fourth-seeded alliance. Led by alliance captain team 16236 Juice (Folsom, California, US), along with partners team 9889 Cruise Control (Flanders, New Jersey, US) and team 16379 KookyBotz (Sammamish, Washington, US), the alliance engineered one of the biggest upsets in the tournament against the number-one seed.

The breakthrough came from the alliance’s extensive manual scouting data. By studying detailed, hand-drawn diagrams of every match, along with match Video-on-Demand replays, the joint scouting team mapped out the exact driving paths, junction preferences, and endgame patterns of their opponents. They even analyzed how the opposing teams reacted to defensive pressure. This deep tactical familiarity allowed the alliance to anticipate every counter-move; despite running technically slower hardware, they executed a hyper-specific strategy that defeated the tournament favorites.

However, the main problem with this approach is how many people it takes. Tracking exact coordinates and paths over a week requires dozens of hours and a huge scouting team. For most medium-sized or smaller teams, this level of data collection is impossible. This bottleneck is exactly what inspired Project

---

\*S. Ray was formerly the Software Division Lead & Vice Captain for *FIRST* Tech Challenge Team Juice #16236. He is currently an undergraduate student with the Department of Electrical Engineering and Computer Sciences, University of California, Berkeley, CA 94720 USA. Contact: [siddharth\\_ray@berkeley.edu](mailto:siddharth_ray@berkeley.edu)

REDACTED, now publicly known as the Juice AutoScout. By automatically pulling coordinate-level match reconstruction from regular match videos, this project aims to bring elite-level tactical data to any team, turning a time-consuming luxury into an accessible tool.

In practice, turning raw match footage into actual field coordinates brings up three main technical challenges:

1. The mapping from video pixels to actual field inches changes depending on where the robot is because field cameras are usually placed at an angle at lower-production-value competitions.
2. Robots constantly bump into each other and/or cross paths from the camera's perspective, turning up to four separate targets into a single, confusing silhouette.
3. Static field borders and colored game pieces look a lot like parts of the robots, creating extra clutter for a tracker.

Off-the-shelf centroid trackers fail quickly in this environment because they cannot handle long periods where robots overlap and swap positions. AutoScout solves this by taking advantage of the specific rules and geometry of an FTC field. Instead of using heavy, slow deep-learning detectors, the system uses a lightweight geometry-first pipeline to keep track of robot identities even through heavy crowding and collisions.

AutoScout was built specifically for this setting. Its objective is to be fine-tuned for a specific set of criteria:

- exactly four logical robot tracks<sup>1</sup>,
- a fixed rectangular field with known physical dimensions,
- a mostly static broadcast or scouting camera,
- frequent physical contact and merged silhouettes,
- colored game pieces and/or game structures that can be mistaken for robots,
- game-specific scoring mechanics and robot-field interactions that are known *a priori*, and
- downstream consumers that need field coordinates rather than image coordinates.

The design philosophy is to exploit these priors explicitly. Instead of treating the scene as a generic unknown video stream, the tracker uses the planarity of the field, the known physical robot scale, the bounded number of robots, and the topological fact that merge events are temporary identity permutations. This sacrifices generality, which limits portability to different robotics competitions (VEX v5, *FIRST* Robotics Competition) or even different FTC game designs, but it yields a system that is easier to tune, explain, and debug than a black-box detector-plus-tracker stack.

Beyond these purely visual challenges, the system's architecture is constrained by the strict operational realities of live competitions. To be truly actionable, match analytics must be delivered immediately in the pits between qualification rounds, leaving no margin for delayed post-processing. Many contemporary computer vision solutions fall back on massive, deep-learning models or large language model (LLM) architectures. However, deploying these black-box models introduces a prohibitive computational overhead, requiring high-end dedicated GPUs that are entirely impractical in a mobile pit environment. To guarantee immediate data availability without relying on cloud infrastructure or heavy local hardware, AutoScout is explicitly designed to be computationally lightweight. By bypassing resource-intensive neural networks in

---

<sup>1</sup>While the system assumes four active tracks during standard gameplay, it handles transient edge cases such as a robot being permanently disabled or removed from the field by dropping the active track count accordingly.

favor of deterministic geometric algorithms, the pipeline minimizes processing latency, enabling high-fidelity trajectory reconstruction to run live on standard, low-power pit laptops.

Conceptually, the system is best understood as two coupled layers. The first layer is the robot tracker, whose job is to recover season-agnostic geometric state: robot positions, visibility, and motion through contact-heavy interactions. The second layer is the shot-analysis module, whose job is to interpret game-specific scoring actions from game-piece motion and game-specific target geometry. Most of this paper focuses on the first layer because it is both the more reusable season-to-season component and the technical core of the project.

## 2 System Objective

Given a calibrated video of a single match, the system estimates a time series of robot states

$$\mathcal{S}_t = \{(x_{i,t}, y_{i,t}, \theta_{i,t}, V_{i,t})\}_{i=0}^3, \quad (1)$$

where  $(x_{i,t}, y_{i,t})$  is robot position on the field,  $\theta_{i,t}$  is motion-derived heading, and  $V_{i,t}$  is a visibility indicator. The public field coordinate system is center-origin on a  $144 \times 144$  inch FTC field, so the center is  $(0, 0)$  and the corners are approximately  $(\pm 72, \pm 72)$  inches.

For mathematical clarity, it is useful to write a fuller latent state for robot  $i$  at frame  $t$  as

$$\mathbf{z}_{i,t} = [x_{i,t} \ y_{i,t} \ \dot{x}_{i,t} \ \dot{y}_{i,t} \ u_{i,t} \ v_{i,t} \ c_{i,t}]^\top, \quad (2)$$

where  $(x_{i,t}, y_{i,t})$  are field coordinates,  $(\dot{x}_{i,t}, \dot{y}_{i,t})$  are field velocities,  $(u_{i,t}, v_{i,t})$  are image coordinates, and  $c_{i,t} \in \mathbb{N}$  is the coast counter measuring how many consecutive frames the track has gone unmatched. The implemented tracker is not a full Bayesian filter, but it behaves like a deterministic stagewise estimator over these latent states.

Let  $\mathcal{Y}_t = \{\mathbf{y}_{j,t}\}_{j=1}^{m_t}$  denote the candidate detections extracted from the current frame. At a high level, the per-frame tracking loop treats state estimation as a deterministic, sequential pipeline rather than a joint optimization framework. Data streams through the system chronologically: the raw video frame is first transformed by a static planar homography to establish spatial scale, which directly parameterizes the downstream morphology and filtering masks. The frame then passes through an anisotropically padded field mask to isolate moving silhouettes and suppress high-frequency false positives from game pieces. Once the cleaned foreground contours are established, the system uses a localized distance-transform peak extractor to resolve merged robot silhouettes into independent sub-center candidates. These candidates are then evaluated against constant-velocity kinematic predictions in a linear assignment cost matrix, which balances spatial displacement with visual appearance descriptors. Finally, the optimized pairings are checked by the Hungarian algorithm and handed off to a multi-frame permutation tracker that handles contact-heavy multi-robot collisions. In practice, decomposing the state-estimation problem into this modular architecture guarantees low computational overhead while isolating seasonal game-specific logic from the stable geometric tracking core.

### 2.1 Two-Part Architecture

At the system level, AutoScout should be understood as two different estimators built on top of the same calibrated video:

1. a robot-tracking subsystem that maps video to four persistent robot trajectories in field coordinates, and
2. a shot-analysis subsystem that uses image-space game-piece motion and target geometry to infer game events such as makes and misses.

The robot tracker is the more reusable core. Its assumptions are tied mainly to the existence of a planar field, a small fixed number of robots, and persistent occlusion-heavy interactions, all of which tend to recur from FTC season to season even when the game objectives change. The shot-analysis layer is different: it depends on the color, motion pattern, and scoring semantics of the current game’s scoring objects and targets. As a result, the tracking core is the part most likely to transfer forward with only retuning, whereas shot detection generally has to be redesigned or substantially reworked for the specific game being played.

### 3 Field Geometry and Calibration

#### 3.1 Projective field model

The geometric foundation is the standard planar homography model [1]. Let image points be written in homogeneous coordinates as  $\tilde{\mathbf{u}} = (u, v, 1)^\top$  and field-plane points as  $\tilde{\mathbf{x}} = (x, y, 1)^\top$ . Because the FTC playing surface is treated as planar, a  $3 \times 3$  homography  $H$  maps image points to the internal corner-origin field plane:

$$\tilde{\mathbf{x}} \sim H\tilde{\mathbf{u}}. \quad (3)$$

Given four corner correspondences  $(\tilde{\mathbf{u}}_k, \tilde{\mathbf{x}}_k)$ , each pair contributes the cross-product constraint

$$\tilde{\mathbf{x}}_k \times (H\tilde{\mathbf{u}}_k) = \mathbf{0}, \quad (4)$$

which yields a linear system in the entries of  $H$  up to scale. Conceptually, the calibration step solves

$$\min_H \sum_{k=1}^4 \|\tilde{\mathbf{x}}_k \times (H\tilde{\mathbf{u}}_k)\|_2^2, \quad (5)$$

after which  $H$  is normalized so that scale is fixed. The inverse homography  $H^{-1}$  is then used whenever a field-plane point must be projected back into image space.

Calibration uses four ordered field corners. The destination plane is fixed to

$$(0, 0), (144, 0), (144, 144), (0, 144),$$

in inches. Public center-origin coordinates are then obtained by shifting:

$$x_c = x - 72, \quad y_c = y - 72.$$

This split between internal and public coordinates matters. Internally, all homography calculations, contour projections, and merge re-anchoring operations occur in the corner-origin frame  $[0, 144]^2$ . Only exported analytics interfaces shift to center-origin coordinates. Retaining the legacy corner-origin frame avoids repeatedly translating geometry routines that are easier to write against a positive rectangular domain.

#### 3.2 Practical calibration path

In the preferred workflow, the user generates `field_corners.json` with an interactive corner-selection tool. The tracker also supports automatic field detection by locating a large quadrilateral through blur, edge detection, contour extraction, and polygon approximation, but this fallback is less reliable than manual corners and is best viewed as a convenience rather than the primary calibration method.

### 3.3 Perspective-derived robot scale

The homography is used for more than coordinate conversion. It also supplies a local image-scale estimate for an 18-inch robot footprint, which serves as a proxy for the maximum allowed robot size envelope<sup>2</sup> in FTC. Let  $\pi(q_1, q_2, q_3) = (q_1/q_3, q_2/q_3)$  denote homogeneous deprojection. Around a field point  $\tilde{\mathbf{x}}$ , the image-space size induced by a one-robot offset in the field frame is approximated by finite differences:

$$\begin{aligned} s_x(\mathbf{x}) &= \left\| \pi(H^{-1}\tilde{\mathbf{x}}) - \pi(H^{-1}(\tilde{\mathbf{x}} + [18, 0, 0]^\top)) \right\|_2, \\ s_y(\mathbf{x}) &= \left\| \pi(H^{-1}\tilde{\mathbf{x}}) - \pi(H^{-1}(\tilde{\mathbf{x}} + [0, 18, 0]^\top)) \right\|_2. \end{aligned}$$

The implementation evaluates this near field center and uses

$$s_{\max} = \max\{s_x, s_y\}$$

as a characteristic pixel footprint for one robot. This single quantity drives several later choices: the neck-breaking kernel radius, the reassociation window, the relaxed split spacing, the appearance-crop size, and post-merge continuation gates. In effect, calibration becomes a scene-specific scale prior for the entire tracker.

## 4 Background Model and Foreground Segmentation

### 4.1 Median background estimation

Let  $I_t(u, v)$  denote the RGB value at frame  $t$  and pixel  $(u, v)$ . The tracker samples  $N = 80$  frames approximately uniformly across the video and computes the per-pixel median:

$$B(u, v) = \text{median}\{I_{t_k}(u, v)\}_{k=1}^N. \quad (6)$$

The median suppresses moving robots much more effectively than a single reference frame and is more robust than a mean when the occupancy pattern is sparse but not negligible.

### 4.2 Field masking and raw foreground

Foreground segmentation begins with background subtraction:

$$\mathbf{D}_t(u, v) = |I_t(u, v) - B(u, v)|, \quad (7)$$

where the absolute value is applied channelwise. After grayscale conversion  $G(\cdot)$  and thresholding at a fixed level  $\tau_D$ , the raw binary foreground is

$$F_t(u, v) = \mathbf{1}[G(\mathbf{D}_t(u, v)) > \tau_D] \mathbf{1}[(u, v) \in \Omega_{\text{field}}], \quad (8)$$

where  $\Omega_{\text{field}}$  is the calibrated field polygon in image space.

The field polygon is not used verbatim. In code, the ordered calibration quadrilateral is padded anisotropically before rasterization. The side, top, and bottom pads are chosen separately so that robots hugging the far wall remain inside the mask while spectator bleed-through and scoreboard artifacts are reduced near the frame boundary. This is a pragmatic correction for real broadcast framing rather than a geometric necessity.

---

<sup>2</sup>This paper was written for the 2025-26 FTC season DECODE, which disallowed expansion past this 18-inch cube until the endgame period, where robots were allowed to vertically expand. Robots typically stopped moving during and after their expansion, so the tracker was typically not affected.

### 4.3 Morphology and neck breaking

The tracker stabilizes the foreground mask with a fixed elliptical morphology kernel:

$$F_t^{(1)} = O_{r_1}(C_{r_1}(F_t)),$$

where  $C$  and  $O$  denote closing and opening. The implementation uses a diameter-7 elliptical kernel at this stage.

This is followed by a perspective-aware erosion-dilation sequence:

$$F_t^{(2)} = \mathcal{D}_{r_2}(\mathcal{E}_{r_2}(F_t^{(1)})),$$

where  $r_2$  is derived from the estimated robot pixel footprint, approximately

$$r_2 \approx 0.15 s_{\max},$$

with a minimum radius of 3 pixels. This second stage is the *neck-breaking* operator. Its purpose is not just denoising: if two robots are connected by a narrow bridge of foreground pixels, the erosion removes that bridge and the subsequent dilation restores the larger bodies without reconnecting them<sup>3</sup>.

### 4.4 Ball suppression

Game pieces form high-frequency false positives for robot tracking, so the system constructs an HSV mask for green and purple ball colors. Let  $M_t^{\text{ball}}$  denote the cleaned ball mask. The final robot-support mask is

$$R_t = F_t^{(2)} \wedge \neg \mathcal{D}_{r_b}(M_t^{\text{ball}}), \quad (9)$$

where a small dilation radius  $r_b$  gives the suppression mask a safety margin around each ball.

However, this ball mask is not a raw color threshold. The implementation applies field masking, opening, closing, projected-area filtering, and dilation before subtraction. The projected-area test is particularly useful because a similarly colored background element may be large in image pixels yet implausible as a ball once projected to the field plane. Reusing the cleaned ball mask in both robot suppression and later shot detection makes the tracker internally consistent about what constitutes ball evidence.

Figure 1 shows the major intermediate layers of this pipeline on a representative frame from `examples/joosmatch.mp4`. Reading left-to-right and top-to-bottom, the figure makes the intended information flow explicit: static scene estimation, motion isolation, field restriction, ball suppression, final robot support extraction, and contour splitting.

## 5 Perspective-Aware Blob Analysis

### 5.1 Contour filtering

After segmentation, the tracker extracts external contours and rejects implausible ones using:

- minimum image-space area,
- minimum projected field area, and
- minimum distance-transform thickness.

---

<sup>3</sup>In this context, “neck breaking” refers to an aggressive localized erosion or watershed-based topological separation that disconnects two weakly joined components at their narrowest shared boundary.

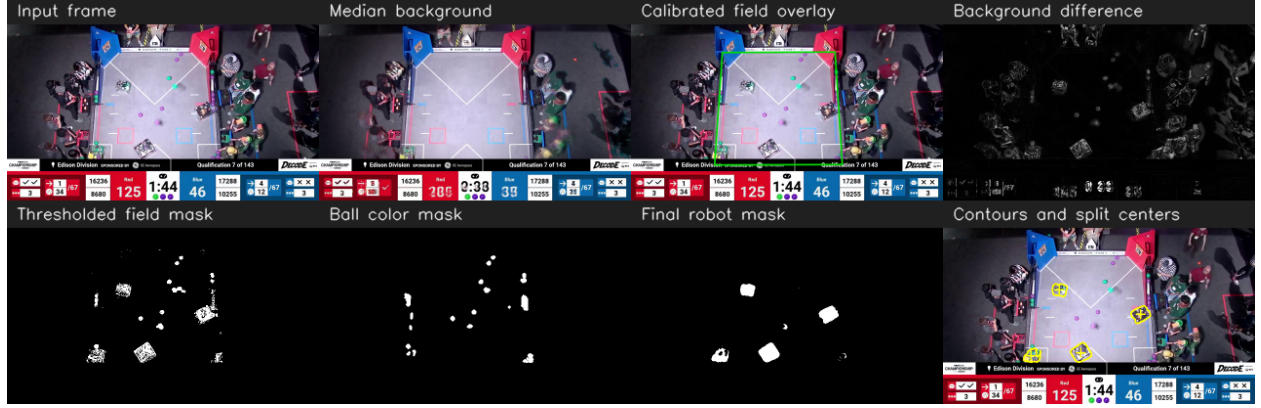


Figure 1: Computer-vision layers generated from `examples/joosmatch.mp4` using the same calibrated tracker pipeline described in the paper. The panels show the input frame, median background, calibrated field overlay, background-difference image, thresholded field-restricted foreground, ball color mask, final robot mask after morphology and ball suppression, and accepted contours with split centers.

If  $A_{\text{img}}(C)$  is contour area in pixels and  $A_{\text{field}}(C)$  is the area of its homography-projected footprint, the acceptance test is of the form

$$A_{\text{img}}(C) \geq \tau_{\text{img}}, \quad A_{\text{field}}(C) \geq \tau_{\text{field}}, \quad \delta_{\text{max}}(C) \geq \tau_{\delta},$$

where  $\delta_{\text{max}}(C)$  is the maximum Euclidean distance-transform thickness inside the filled contour.

These thresholds protect against different failure modes. The image-area threshold rejects speckle and crowd leakage. The projected-area threshold protects far-field robots that may be small in pixel count but still occupy plausible field area after projection. The thickness threshold rejects long, thin artifacts that survive closing/opening but are too filamentary to represent robot support regions.

## 5.2 Parent contours and sub-detections

Each accepted contour is treated as a *parent region*. The tracker does not assume one parent corresponds to one robot. Instead, it estimates a set of sub-centers

$$C(C) = \{\mathbf{c}_1, \dots, \mathbf{c}_{n_C}\}$$

inside the parent, where  $n_C$  may exceed one. Every sub-center inherits the same parent contour identifier and contour-level quality metadata, but corresponds to a distinct candidate detection. This representation is central: the assignment stage can allow several tracks to bind to different sub-centers from the same parent contour, and merge handling can reason explicitly about which tracks share a common support region.

## 6 Split-Center Estimation for Merged Robots

### 6.1 Distance-transform peak extraction

Each parent contour may contain one or more robots. To estimate sub-centers inside a merged blob, the tracker fills the contour into a binary mask and computes its Euclidean distance transform. Peaks of this transform approximate centers of maximal inscribed disks and therefore tend to sit near robot interiors.

Let  $M$  denote the filled contour mask and let  $\delta(\cdot)$  denote the distance transform. Candidate peaks are first extracted by thresholding

$$\delta(\mathbf{p}) \geq \alpha \max_{\mathbf{q}} \delta(\mathbf{q}), \quad (10)$$

with a strict ratio  $\alpha = 0.40$ , followed by erosion and connected-components analysis. Equivalently, if

$$P_\alpha(M) = \{\mathbf{p} \in M : \delta(\mathbf{p}) \geq \alpha \delta_{\max}\}, \quad \delta_{\max} = \max_{\mathbf{q} \in M} \delta(\mathbf{q}), \quad (11)$$

then each sub-center is the centroid of one connected component of  $P_\alpha(M)$  after erosion.

## 6.2 Expected robot count from contour area

The tracker estimates how many robots are likely represented by a contour by comparing contour area against a one-robot scale proxy

$$A_1 \approx s_{\max}^2.$$

If

$$r(C) = \frac{A_{\text{img}}(C)}{A_1}, \quad (12)$$

the expected robot count is computed conservatively as

$$\hat{n}(C) = \begin{cases} 4, & r(C) \geq 3.5, \\ 3, & r(C) \geq 2.6, \\ 2, & r(C) \geq 1.6, \\ 1, & \text{otherwise.} \end{cases} \quad (13)$$

These thresholds deliberately require substantial evidence before the tracker escalates from a singleton to a two-, three-, or four-robot interpretation.

## 6.3 Relaxed splitting

If the strict peak extractor returns fewer centers than  $\hat{n}(C)$ , the tracker retries with a more permissive peak ratio and a smaller separation kernel. For pairwise merges the relaxed threshold remains relatively cautious; for three- and four-robot merges it is more permissive, favoring recall over clean separation. This two-stage procedure can be interpreted as a deterministic coarse-to-fine peak detector: first recover only strong interior modes, then broaden the superlevel set when contour area indicates additional hidden structure.

## 6.4 Augmenting peaks with track priors

When even the relaxed split under-resolves a merge, the tracker injects predicted track positions into the peak set. Let  $\hat{\mathbf{u}}_{i,t|t-1}$  denote the predicted image position of track  $i$ . If  $\hat{n}(C) > |C(C)|$ , then tracks whose predictions lie inside or just outside the contour are eligible to contribute pseudo-centers, provided they are not too close to already extracted peaks. The resulting candidate set is therefore

$$\tilde{C}(C) = C(C) \cup \mathcal{P}(C),$$

where  $\mathcal{P}(C)$  is a small set of track-conditioned pseudo-observations.

This is a crucial design choice. In a dense merge, the absence of enough current peaks should not imply the absence of robots. By allowing state-conditioned pseudo-centers, the tracker preserves the hypothesis that multiple robots are still present even when the current frame cannot fully resolve them from foreground geometry alone.

## 6.5 Under-resolved merge gating

The tracker also records which tracks are likely trapped inside an under-resolved contour. Let  $n_C^{\text{exp}} = \hat{n}(C)$  and let  $n_C^{\text{split}}$  be the number of extracted or augmented sub-centers. If

$$n_C^{\text{exp}} > n_C^{\text{split}},$$

then any track whose predicted image position lies inside or near that contour is marked under-resolved. For such a track, the later assignment stage refuses matches to blobs that are inconsistent with that same merge support. This prevents a robot that is probably hidden in a contact cluster from snapping to an unrelated singleton elsewhere in the frame merely because that singleton is locally cheap under a pure distance metric.

Let  $\mathcal{U}_t$  denote the set of under-resolved tracks at frame  $t$ , and let  $C_j$  be the parent contour of candidate  $j$ . If  $\text{sdist}(C_j, \hat{\mathbf{u}}_{i,t|t-1})$  is the signed image-space distance from the predicted position to the contour boundary, define the contour-consistency gate

$$g_{ij}^{\text{merge}} = \mathbf{1} \left[ \left| \text{sdist}(C_j, \hat{\mathbf{u}}_{i,t|t-1}) \right| \leq 0.75 s_{\max} \right].$$

Then under-resolved tracks use

$$C_{ij} \leftarrow \begin{cases} C_{ij}, & i \notin \mathcal{U}_t \text{ or } g_{ij}^{\text{merge}} = 1, \\ +\infty, & i \in \mathcal{U}_t \text{ and } g_{ij}^{\text{merge}} = 0. \end{cases}$$

This rule does not alter the skip-column penalty itself; it removes contour-inconsistent real assignments so that the optimizer either stays attached to the same merge support or explicitly skips the track for that frame.

## 7 Track State and Motion Model

Each logical robot track stores

- field position,
- image position,
- smoothed field velocity,
- smoothed image velocity,
- coast count, and
- visibility state.

The deterministic motion model is constant-velocity in both field and image coordinates. Writing

$$\mathbf{x}_{i,t} = \begin{bmatrix} x_{i,t} \\ y_{i,t} \end{bmatrix}, \quad \mathbf{v}_{i,t} = \begin{bmatrix} \dot{x}_{i,t} \\ \dot{y}_{i,t} \end{bmatrix}, \quad (14)$$

prediction is

$$\hat{\mathbf{x}}_{i,t+1|t} = \mathbf{x}_{i,t} + h_{i,t} \mathbf{v}_{i,t}, \quad (15)$$

with horizon

$$h_{i,t} = \min(1 + 0.15 c_{i,t}, 2.0). \quad (16)$$

An analogous predictor is maintained in image coordinates:

$$\hat{\mathbf{u}}_{i,t+1|t} = \mathbf{u}_{i,t} + h_{i,t} \dot{\mathbf{u}}_{i,t}. \quad (17)$$

If a field observation  $\mathbf{y}_{i,t+1}^{\text{field}}$  is accepted, the state position is updated directly:

$$\mathbf{x}_{i,t+1} = \mathbf{y}_{i,t+1}^{\text{field}}.$$

Velocity is updated by exponential smoothing:

$$\mathbf{v}_{i,t+1} = \beta \mathbf{v}_{i,t} + (1 - \beta) (\mathbf{x}_{i,t+1} - \mathbf{x}_{i,t}), \quad \beta = 0.45.$$

The raw displacement is clipped to a permissive per-frame field-displacement cap before smoothing:

$$\|\mathbf{x}_{i,t+1} - \mathbf{x}_{i,t}\|_2 \leq v_{\max}, \quad v_{\max} = 140 \text{ in/frame}.$$

At the processed frame rates used by AutoScout, this threshold is intentionally well above any physically realizable FTC robot displacement over one processed frame. It therefore acts as a loose outlier clamp against bad projections or accidental reassociation jumps, not as a tight drivetrain model.

One implementation detail is easy to misstate: when no measurement is accepted, the tracker uses prediction for gating and assignment, but it does *not* commit the predicted position into the persistent state. Instead, it keeps the last accepted pose and increments the coast counter:

$$c_{i,t+1} = c_{i,t} + 1.$$

Thus coasting represents uncertainty about visibility, not dead-reckoned extrapolation of the public trajectory. A track remains visible only for a short grace period while unmatched:

$$V_{i,t} = \mathbf{1}[c_{i,t} \leq c_{\text{vis}}],$$

with  $c_{\text{vis}} = 8$  in the implementation.

Heading is not estimated from robot shape. Instead, it is motion-derived:

$$\theta_{i,t} = \begin{cases} \theta_{i,t-1}, & \|\mathbf{v}_{i,t}\| < \varepsilon, \\ \text{atan2}(\dot{y}_{i,t}, \dot{x}_{i,t}), & \text{otherwise.} \end{cases}$$

This is simple and stable, though it cannot recover robot orientation when a robot strafes or rotates in place.

## 7.1 Relation to Kalman filtering

Kalman filters are very commonly used in FTC and were a starting point for the first iteration of the predictor system. This predictor-update structure is intentionally Kalman-like in shape but not a true Kalman filter in the classical linear-Gaussian sense. A standard constant-velocity Kalman filter would write

$$\mathbf{z}_{i,t+1} = \mathbf{A}\mathbf{z}_{i,t} + \mathbf{w}_{i,t}, \quad \mathbf{y}_{i,t} = \mathbf{C}\mathbf{z}_{i,t} + \mathbf{n}_{i,t},$$

with Gaussian process noise  $\mathbf{w}_{i,t}$  and observation noise  $\mathbf{n}_{i,t}$ , and would propagate both a state mean and covariance through an optimal linear update [2]. AutoScout does not maintain that probabilistic covariance state. Instead, it uses deterministic prediction, hard gating<sup>4</sup>, hand-designed costs, and explicit combinatorial logic for data association and merge resolution.

This distinction matters because the dominant failure mode in FTC footage is usually not small Gaussian localization noise but prolonged identity ambiguity caused by occlusion, contact, and blob fusion. In this problem setting, identity preservation is governed more by association and ordering constraints than by optimal single-track state estimation.

<sup>4</sup>While traditional Kalman filters scale gating thresholds dynamically using the measurement residual covariance  $S$ , AutoScout achieves spatial scaling adaptively by projecting fixed physical boundaries into image space via the perspective scale factor  $s_{\max}$ .

## 8 Assignment and Re-Identification

### 8.1 Candidate representation

Every split center becomes a candidate record containing field location, image location, parent contour identifier, contour-derived quality penalties, split multiplicity, and optionally an appearance descriptor. Formally, a candidate can be written as

$$\mathbf{y}_{j,t} = (\mathbf{x}_{j,t}^{\text{field}}, \mathbf{x}_{j,t}^{\text{img}}, \kappa_{j,t}, \mathbf{h}_{j,t}),$$

where  $\kappa_{j,t}$  collects contour-quality metadata and  $\mathbf{h}_{j,t}$  is an optional appearance histogram.

The parent contour identifier is not bookkeeping noise; it is a first-class signal. If two candidates share the same parent, then they are not independent detections but alternative substructure hypotheses inside one merged region. Several later rules, especially merge discovery and under-resolved gating, are defined directly in terms of this parent relation.

### 8.2 Assignment cost

Assignment is solved between four persistent logical tracks and the current candidate set. The implementation builds a cost matrix whose principal terms are

$$C_{ij} = d_{ij}^{\text{field}} + 0.35 d_{ij}^{\text{img}} + q_j + \lambda_a a_{ij} + \lambda_\ell \ell_{ij}, \quad (18)$$

where:

- $d_{ij}^{\text{field}}$  is normalized field-space distance from predicted track  $i$  to candidate  $j$ ,
- $d_{ij}^{\text{img}}$  is normalized image-space distance,
- $q_j$  is a contour-quality penalty,
- $a_{ij}$  is a re-identification mismatch against optional reference histograms, and
- $\ell_{ij}$  represents post-merge lock penalties when active.

More explicitly, the normalized distance terms are

$$d_{ij}^{\text{field}} = \frac{\|\mathbf{x}_{j,t}^{\text{field}} - \hat{\mathbf{x}}_{i,t|t-1}\|_2}{\tau_i^{\text{field}}}, \quad d_{ij}^{\text{img}} = \frac{\|\mathbf{x}_{j,t}^{\text{img}} - \hat{\mathbf{u}}_{i,t|t-1}\|_2}{1.25 s_{\text{max}}}, \quad (19)$$

where

$$\tau_i^{\text{field}} = \begin{cases} 30, & c_{i,t} \leq c_{\text{vis}}, \\ 144, & c_{i,t} > c_{\text{vis}}. \end{cases} \quad (20)$$

The reacquisition radius therefore expands substantially once a track has been invisible beyond the short visible-coast window.

If  $A_C$  is the parent contour area and  $\bar{A}_C = A_C/n_C$  is the per-split area, the quality term behaves like

$$q_j \approx 0.28 \left| \log \left( \frac{\bar{A}_C}{A_1} \right) \right| + \mathbf{1}[n_C = 1 \wedge A_C > 1.8 A_1] \eta(A_C),$$

where  $\eta(\cdot)$  increases with oversized unsplit contours. The intent is that a candidate supported by an implausibly large singleton contour should be treated with suspicion because it may hide unresolved robots.

### 8.3 Hard gating and skip decisions

Before optimization, the tracker rejects impossible assignments through hard gates. These include:

- excessive field-space displacement,
- excessive image-space displacement relative to perspective-scaled robot size,
- implausibly poor singleton candidates for tracks that recently exited a merge,
- post-merge continuation violations, and
- under-resolved-track assignments to blobs inconsistent with the same merge support.

If  $\Pi_t \in \{0, 1\}^{4 \times (m_t + 4)}$  is the assignment matrix after adding four skip columns, the combinatorial problem is

$$\min_{\Pi_t} \sum_{i=0}^3 \sum_{j=1}^{m_t+4} C_{ij} \Pi_{ij,t}, \quad (21)$$

subject to

$$\sum_{j=1}^{m_t+4} \Pi_{ij,t} = 1 \quad \text{for each track } i, \quad \sum_{i=0}^3 \Pi_{ij,t} \leq 1 \quad \text{for each real candidate } j. \quad (22)$$

The skip columns are not assigned one universal constant. Their penalty increases modestly with coast duration, is softened for recently merged tracks, and is hardened when a post-merge lock is active. This gives the solver a mathematically explicit way to express three qualitatively different situations: a short benign miss, a prolonged lost track, and a track that ought to have a continuation because it has just separated from contact.

### 8.4 Optimization

The preferred solver is the Hungarian algorithm via SciPy [3]. If SciPy is unavailable, usually due to compatibility issues, the code falls back to a greedy minimum-cost matching routine. Because the problem size is fixed at four tracks, even the fallback remains fast enough for practical use, though the Hungarian solver is the more principled optimizer.

### 8.5 Appearance model

The tracker takes a visual "fingerprint" of each tracked robot to help break ties during visually ambiguous merge-splits. Appearance is represented by a hue-saturation histogram extracted from an HSV crop near the candidate center, optionally contour-masked. If  $\mathbf{h}_i$  and  $\mathbf{h}_j$  are normalized histograms, the Bhattacharyya coefficient [4] is

$$\rho(\mathbf{h}_i, \mathbf{h}_j) = \sum_b \sqrt{h_i^{(b)} h_j^{(b)}}, \quad (23)$$

and the appearance mismatch is taken as

$$a_{ij} = \sqrt{1 - \rho(\mathbf{h}_i, \mathbf{h}_j)}. \quad (24)$$

The implementation realizes this using an  $18 \times 16$  hue-saturation histogram over a circular crop whose radius is approximately  $0.55 s_{\max}$ . When a contour is available, the crop is further intersected with the contour mask so that background pixels do not dominate the feature. The main appearance signal comes from optional supervised references obtained from a manually labeled pose file. Those references are sampled back from

the video at fixed stride and stored per robot; assignment then uses the minimum Bhattacharyya distance to that robot’s reference bank.

Although the crop is converted to HSV, the descriptor intentionally uses only hue and saturation bins and omits the value channel. This reduces sensitivity to venue-specific brightness swings and shadows, though it does not make the feature fully illumination invariant.

This model is intentionally lightweight. It is not strong enough to replace geometry, but it is often strong enough to break ties during reacquisition or after a visually ambiguous split. In addition, a separate appearance snapshot is stored inside post-merge locks, which lets the tracker penalize continuations that are inconsistent with what the robot looked like at merge exit.

## 9 Initialization and Special Suppression Rules

### 9.1 Bootstrap initialization

Before normal tracking starts, the system waits for a plausible four-candidate scene and chooses the best left-to-right lineup from a limited candidate pool. If the top candidates are  $\mathcal{B} = \{b_1, \dots, b_M\}$ , the bootstrap considers combinations of four candidates, sorts each combination by field  $x$  position, and scores

$$J_{\text{init}} = \sum_{r=1}^4 q_{b_r} + \lambda_s \sum_{r=1}^3 \max(0, \Delta_{\min} - (x_{r+1} - x_r))^2.$$

The spacing term discourages degenerate initializations in which multiple tracks are seeded at nearly the same horizontal location. The best-scoring lineup becomes robots 0 through 3. User-supplied initial field positions can bypass this bootstrap entirely.

### 9.2 Static-blob suppression

Persistent non-robot blobs are tracked coarsely in image space and suppressed if they remain approximately stationary over a configured temporal window. Candidate centroids are bucketed on a coarse grid and matched against the previous frame’s bucket positions. If a bucket stays within a small motion threshold for many consecutive frames, and no currently visible robot is nearby in field space, future candidates from that bucket are dropped. This rule is intentionally simple: it is a temporal veto against stationary clutter.

### 9.3 Corner-zone suppression

The top corners of the DECODE (2025-26) field tend to produce recurring false blobs from field structures. A dedicated suppression zone in those corners prevents tracks from snapping into static geometry unless a robot is already established nearby and visible. This rule is crude, but it directly targets one of the most frequent persistent false-positive regions in the match footage.

## 10 Identity Preservation Through Merges

### 10.1 Problem formulation

The main difficulty in FTC footage is not simply detection but identity preservation when two or more robots overlap for extended periods. A merged blob can hide multiple centers, and naive trackers often swap identities after separation. Mathematically, the tracker must preserve the mapping between persistent labels and an unordered, partially observed set of split peaks.

## 10.2 Merge groups

After assignment, tracks are grouped by parent contour. Any parent contour containing multiple assigned tracks is treated as a merge. The tracker stores a *merge group* containing the participating track IDs, the merge-entry axis  $\mathbf{a} \in \mathbb{R}^2$ , and additional state that depends on whether the merge involves two robots or more.

In the implementation, the axis is the unit vector joining the pair of entry positions with maximum separation. This is a practical proxy for the dominant elongation direction of the group at merge onset. Using the widest pair, rather than for example a PCA on only four points, gives a stable axis even in small-cardinality cases and aligns the ordering logic with the dimension along which permutations are most visually separable.

## 10.3 Two-robot merges

For two robots, the logic reduces to detecting whether the relative order of the split peaks along the entry axis has crossed. If robots  $p$  and  $q$  entered the merge with projected separation

$$\Delta_0 = \mathbf{a}^\top (\mathbf{x}_{p,t_0} - \mathbf{x}_{q,t_0}),$$

and later have split-center separation

$$\Delta_t = \mathbf{a}^\top (\mathbf{c}_{p,t} - \mathbf{c}_{q,t}),$$

then a sign change  $\Delta_0 \Delta_t < 0$  indicates an ordering reversal. If so, the logical states are swapped on separation; otherwise they are preserved.

## 10.4 Three- and four-robot merges

The distinctive implementation feature of AutoScout is its explicit permutation logic for higher-cardinality merges. At merge entry, the participating tracks are sorted along the entry axis to define an *entry order*. During the merge, the current split peaks are sorted along that same axis to define a *current order*. The relation

$$\text{entry order} \rightarrow \text{current order}$$

encodes the permutation that should be applied when the robots separate.

Let the merge group be  $G = \{i_1, \dots, i_k\}$  with  $k \in \{3, 4\}$ . Define entry-order and current-order permutations by sorting projected positions along  $\mathbf{a}$ . Then the rank-preserving identity map is

$$\mathbf{c}_{\sigma_t(r),t} \longleftrightarrow i_{\sigma_0(r)}, \quad r = 1, \dots, k. \quad (25)$$

This is the core mathematical idea: labels are preserved by rank along a stable merge axis rather than by nearest Euclidean position alone.

Two implementation details are essential:

1. while a 3+ robot merge persists, the tracker continually reassigns newly extracted peaks to the currently tracked merge members and updates their positions from those peaks, and
2. if a frame yields too few peaks, the tracker keeps the previous peak assignment instead of resetting merge state.

These choices prevent long-duration merges from collapsing into stale entry-time state. The merge group therefore behaves like a small internal tracker whose latent variables are the current peak assignment and the evolving order along the merge axis.

## 10.5 Order voting during ambiguous merges

The implementation goes one step beyond a single-frame rank comparison. Each merge group accumulates *order votes*. Whenever a current ordering is observed, each track receives a weighted vote for the slot it occupies. If the current peak extraction is complete, the vote weight is high; if the split is partial, the vote weight is reduced. At separation, the tracker evaluates all permutations of the group and chooses the one with maximum cumulative slot agreement:

$$\sigma^\star = \arg \max_{\sigma \in S_k} \sum_{r=1}^k w_{\sigma(r),r}, \quad (26)$$

where  $w_{i,r}$  is the accumulated evidence that track  $i$  belongs in rank slot  $r$ . Since  $k \leq 4$ , brute-force search over  $S_k$  is trivial. This voting scheme turns noisy framewise order estimates into a more stable multi-frame decision.

## 10.6 Relabeling from exit blobs

When a higher-cardinality merge breaks apart, the first separation frame can still be ambiguous because several nearby blobs compete to continue the merged tracks. The code therefore runs a dedicated relabeling step for 3+ robot merges. Let  $E = (e_1, \dots, e_k)$  be the resolved expected order from the vote accumulator. The tracker gathers a local candidate set around the merge peaks, then tests combinations of  $k$  exit blobs sorted along the merge axis. For a proposed map  $e_r \mapsto b_r$ , the cost includes:

- distance from exit blob to the track’s last merge peak,
- distance to the predicted field position,
- re-identification cost,
- distance to the track’s entry-time appearance feature,
- slot inconsistency penalty, and
- disagreement with any active post-merge lock.

The best valid combination becomes the new continuation map. This is effectively a second local assignment problem specialized to merge exit, where the search space is tiny but the identity stakes are high.

## 10.7 Post-merge locks

After a merge resolves, the tracker stores short-lived post-merge locks containing recent position, velocity, and appearance evidence. These locks bias assignment toward the correct continuation and help prevent immediate re-swaps or duplicate matches after separation. Formally, they act as a transient additive term  $\ell_{ij}$  in the assignment cost, with smaller values for candidates that are consistent with the stored post-merge continuation.

Each lock contains image position, image velocity, field position, field velocity, an optional appearance feature snapshot, and the merge-group key from which it arose. A lock therefore serves two purposes. First, it imposes a continuation prior for the same track in subsequent frames. Second, it allows the tracker to detect and prune duplicate post-merge assignments: if two tracks from the same former merge land on nearly the same continuation blob, the one with worse support cost is dropped. This duplicate-pruning rule directly targets a common failure mode immediately after separation.

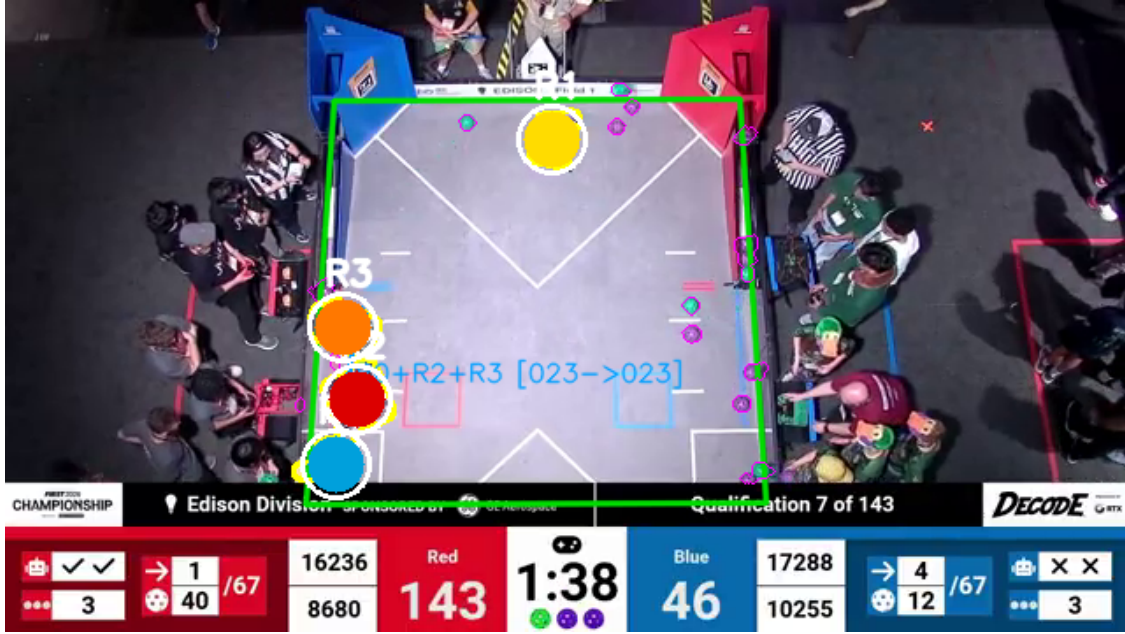


Figure 2: Annotated merge frame from `examples/joosmatch.mp4`. The three-robot merge shown here is exactly the kind of event for which simple centroid tracking becomes unreliable. The tracker therefore reasons over contour-internal peaks and the order of robots along a stable merge axis instead of relying only on nearest-neighbor continuation.

Figure 2 illustrates what the merge logic looks like in a real frame. The green quadrilateral shows the calibrated field support, yellow outlines mark accepted robot contours, white crosses indicate split centers, colored circles indicate the current logical track identities, and the cyan arrow-plus-label encodes the merge-axis ordering information used to preserve identity through the overlap.

## 11 Game-Specific Shot Detection

### 11.1 Why shot detection is separate

The robot tracker reasons on the field plane, but shots involve airborne balls and elevated goal openings. Because a planar homography is inappropriate for projectile motion, shot detection is implemented entirely in image space.

This module should be viewed as game-specific by design. In the current implementation it is tailored to a game with purple and green balls and visually identifiable goal openings, so its thresholds, color masks, launch logic, and resolution rules are tightly coupled to that game’s scoring structure. Unlike the robot tracker, it is not expected to transfer unchanged across FTC seasons.

### 11.2 Goal-opening detection

On setup, the detector uses the median background to locate the large red and blue scoring structures through HSV segmentation. Search is restricted to the upper half of the frame, and each alliance color is constrained to its expected half-field side. After contour cleanup, the largest plausible colored region is selected, its upper

portion is isolated, and a convex hull is shrunk anisotropically to form an effective opening polygon<sup>5</sup>. If this process fails, the detector falls back to coarse opening triangles obtained by projecting hand-chosen field points through the inverse homography.

### 11.3 Ball tracking and launch detection

Ball blobs come from the same green/purple mask used for robot suppression. These blobs are associated frame to frame by nearest-neighbor tracking. Let a ball trajectory in image space be  $\mathbf{b}_\tau = (u_\tau, v_\tau)$  for  $\tau = t_0, \dots, t_1$ . A track is considered a launched shot only if it satisfies several motion and proximity tests:

$$\|\mathbf{b}_{t_0} - \mathbf{r}_{i,t_0}^{\text{img}}\|_2 \leq \tau_{\text{near}},$$

$$\|\mathbf{b}_{t_1} - \mathbf{b}_{t_0}\|_2 \geq \tau_{\text{travel}},$$

$$v_{t_1} - v_{t_0} \leq -\tau_{\text{up}},$$

and

$$(\mathbf{b}_{t_1} - \mathbf{b}_{t_0})^\top (\mathbf{b}_{t_0} - \mathbf{r}_{i,t_0}^{\text{img}}) > 0,$$

where image  $v$  increases downward. These inequalities enforce that the ball starts near a robot, travels far enough, moves upward in the frame, and moves outward from the inferred shooter.

Shooter selection itself is slightly more refined than nearest distance to the first ball point. The detector estimates a small back-projected launch origin using the first few ball displacements, then scores each visible robot by a weighted combination of distance to that origin, distance to the first sample, and a penalty when the ball appears substantially below the robot in the image. This reflects the geometry of FTC footage, where the first cleanly segmented ball pixel is often already downstream from the true release point.

### 11.4 Shot resolution

The outcome logic is heuristic:

- **Made:** the ball track enters a goal-opening polygon with sufficient confirmation and then disappears, which indicates it has gone through the goal on oblique perspective cameras. If the camera is a top-down view, inferred from the field shape, it checks if the ball crosses the detected classifier bar.
- **Missed:** the ball clearly approaches the goal region or reaches the upper field region without valid goal entry.

If  $P_{\text{goal}}$  is the goal-opening polygon and  $\chi_{P_{\text{goal}}}$  is its indicator, a simplified made-shot rule is

$$\sum_{\tau=t_0}^{t_1} \chi_{P_{\text{goal}}}(\mathbf{b}_\tau) \geq \tau_{\text{in}} \quad \text{and} \quad \mathbf{b}_{t_1+1} \text{ is absent.}$$

The detector also deduplicates shot events in time on a per-robot basis, which prevents a single physical launch from producing multiple events when the ball track fragments briefly. This does not affect multiple balls being shot by a single robot at once. The stored shot location is the shooter's field position at launch time rather than the ball position, which is more useful for downstream tactical analysis.

---

<sup>5</sup>This directional shrinkage creates a conservative spatial boundary, ensuring that slight camera vibrations or physical field assembly tolerances do not cause bouncing balls to trigger false positive scoring events.

## 12 Limitations

The system remains heuristic and makes several strong assumptions:

- the camera is mostly static,
- the field corners are accurate,
- the field is planar,
- four logical tracks are always sufficient,
- robot centers can be represented by one point on the field plane, and
- color thresholds remain reasonably stable under venue lighting.

Several failure modes follow directly from these assumptions. A homography cannot model camera motion, rolling-shutter distortions, or the vertical extent of a robot body. The segmentation pipeline can fail when venue lighting shifts the apparent ball or goal hues outside the tuned HSV bands. Distance-transform splitting assumes that robot interiors create separable thickness peaks; this can break under extreme full-overlap occlusion or if a large field element becomes connected to the robots. Motion-derived heading is only a kinematic proxy, so strafing or in-place rotation are not distinguishable. The shot detector is also intentionally weaker than the robot tracker: it has no 3D ballistic model and resolves outcomes entirely from image-space heuristics.

The appearance model is another bounded-strength component. Because it uses low-dimensional HSV histograms, it can help only when robots are visually distinct enough at the crop scale being measured. It is best understood as a tie-breaker layered on top of geometry, not as a standalone re-identification system.

The season-to-season reuse boundary follows naturally from these limitations. The robot tracker mostly depends on scene geometry and multi-robot interaction structure, so it is plausible to carry it season to season with calibration changes and modest heuristic retuning. The shot-analysis layer, or better yet scoring-analysis, depends on the current game’s scoring mechanics, object appearance, and target geometry, so it should be treated as a per-game module rather than a permanent universal component.

## 13 Discussion and Broader Impact

Following the initial public release of AutoScout to the FTC community, a significant secondary use case emerged: using the pipeline core as a low-cost, automated scoring-verification system. Historically, automated scoring systems, such as those used in the *FIRST* Robotics Competition (FRC), have often been viewed as impractical for FTC because of infrastructure and component costs. Consequently, FTC has continued to rely heavily on manual volunteer scorekeeping. In high-scoring matches, that manual process can become error-prone, and current event procedures may leave teams with little practical recourse when a scoring mistake is recorded. Resource-constrained teams that invest months of engineering labor and substantial financial resources into their robots can therefore be eliminated because of uncorrectable human logging errors. By leveraging the lightweight vision core of AutoScout, but simplifying it for score tracking, higher-level events could plausibly implement real-time, low-overhead scoring audits without expensive sensor hardware by piggybacking on existing broadcast infrastructure. This shifts the utility of the system from a pure tactical analytics engine to an accessible tool for competitive equity.

## 14 Conclusion

AutoScout is best understood as a two-part system combining to form the complete analysis of a game: a reusable geometric robot tracker and a game-specific shot-analysis layer. Its core contribution is the robot-tracking half, which combines a field homography, perspective-scaled morphology, contour-internal peak extraction, explicit skip-aware assignment, and a merge model that preserves identity through order and permutation rather than proximity alone. In this application domain, that merge logic is the central algorithmic idea: once multiple robots inhabit one silhouette, the problem is no longer merely “where is the blob center?” but “which persistent label should emerge from which side of the blob?”

The broader lesson of the system is methodological. In a narrow but structurally rich vision problem, a transparent stack of geometric transforms, morphological operators, and small combinatorial solvers can outperform more generic formulations simply because the failure modes are the right ones to optimize for. Here, the difficult part is not generic object detection but identity preservation under perspective and contact, and that part is likely to remain useful across seasons. By contrast, event logic tied to a specific scoring object or target should be expected to change with the game. AutoScout therefore separates a relatively stable geometric core from a more disposable game-interpretation layer.

## References

- [1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2004.
- [2] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Trans. ASME, J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, Mar. 1960.
- [3] J. Munkres, “Algorithms for the assignment and transportation problems,” *J. Soc. Ind. Appl. Math.*, vol. 5, no. 1, pp. 32–38, Mar. 1957.
- [4] A. K. Bhattacharyya, “On a measure of divergence between two statistical populations defined by their probability distributions,” *Bull. Calcutta Math. Soc.*, vol. 35, pp. 99–109, 1943.